AIR QUALITY ANALYSIS AND PREDICTION USING DEEP LEARNING

PROJECT REPORT

Submitted by

Deepak Mouriya M

Register Number: 24AD026

Submitted to

Department of Artificial Intelligence and Data Science

KPR Institute of Engineering and Technology

Course: U21ADP05 – Exploratory Data Analysis and Visualization

Assignment: Assignment II

Academic Year: 2026–2027

Batch: 2024–2028

Year/Semester: V

ABSTRACT

This project presents an exploratory analysis and deep learning-based regression study of air quality measurements using the Air Quality dataset obtained from the UCI Machine Learning Repository. The original dataset contains 9,358 observations and 15 attributes representing hourly pollutant measurements, sensor responses, and environmental variables collected in an urban area of Italy during 2004–2005. The workflow includes data inspection, missing-value handling, duplicate checks, exploratory data analysis, visualization, numerical preprocessing, and chronological train-validation-test splitting. Six visualizations are used to examine pollutant distributions, potential outliers, correlations, and hourly variations. A Multilayer Perceptron (MLP) regression model is developed using TensorFlow/Keras to estimate carbon monoxide concentration, represented by CO(GT), from other numerical features. Model performance is evaluated using Mean Absolute Error, Root Mean Squared Error, and the coefficient of determination. The model achieves a test MAE of 0.4542, RMSE of 0.6097, and R^2 of 0.7861. These results demonstrate the potential of deep learning for analysing air quality measurements.

Keywords: Air quality, exploratory data analysis, data visualization, deep learning, MLP, regression, carbon monoxide, TensorFlow, UCI Machine Learning Repository.

1. INTRODUCTION

Air pollution is one of the major environmental challenges affecting human health, ecosystems, and quality of life. Pollutants such as carbon monoxide (CO), nitrogen oxides (NOx), nitrogen dioxide (NO_2), and benzene are associated with various sources, including transportation, industrial activities, and fuel combustion.

Air quality monitoring systems collect large amounts of data through sensors and reference measurement instruments. Analysing these measurements helps identify pollutant distributions, understand relationships between variables, and examine variations over time.

Exploratory Data Analysis (EDA) is an essential step in understanding a dataset before applying machine learning or deep learning techniques. It involves inspecting data quality, examining statistical summaries, identifying missing values and potential outliers, and visualizing relationships among variables.

This project applies EDA, data preprocessing, visualization, and deep learning to the Air Quality dataset obtained from the UCI Machine Learning Repository. A Multilayer Perceptron (MLP) regression model is developed to estimate carbon monoxide concentration using other available numerical features.

The project focuses on estimating CO(GT) from the available measurements. It does not directly predict future air quality or establish the causes of pollution.

1.1 Problem Statement

Air quality datasets contain multiple pollutant measurements and environmental variables that may have complex relationships. Missing sensor readings, unusual observations, and differences in feature scales can make data analysis and model development challenging.

The problem addressed in this project is to explore air quality measurements, identify meaningful patterns through visualization, preprocess the data, and develop a deep learning regression model to estimate carbon monoxide concentration from other numerical attributes.

1.2 Objectives

- To analyse a real-world air quality dataset containing multiple attributes.
- To inspect data types, descriptive statistics, missing values, and duplicate records.
- To identify potential outliers and understand pollutant distributions.
- To create and interpret at least five meaningful visualizations.
- To preprocess numerical features using appropriate imputation and scaling techniques.
- To develop an MLP regression model using TensorFlow/Keras.
- To evaluate the model using MAE, RMSE, and R^2.
- To examine feature importance and compare alternative model configurations.
- To identify limitations and suggest future improvements.

1.3 Scope of the Project

The scope of this project is limited to analysing the available Air Quality dataset and estimating the CO(GT) target variable using other numerical measurements. The analysis demonstrates an end-to-end data science workflow, from dataset acquisition and EDA to model training and evaluation.

The project is an educational prototype. Its results should not be treated as a substitute for official air quality monitoring, environmental measurements, or future pollution forecasting.

2. DATASET DESCRIPTION

The dataset used in this project is the Air Quality dataset available from the UCI Machine Learning Repository.

Dataset source: https://archive.ics.uci.edu/dataset/360/air+quality

DOI: https://doi.org/10.24432/C59K5F

The dataset contains hourly air quality measurements collected in an urban area of Italy from March 2004 to April 2005. It includes pollutant concentrations, sensor responses, temperature, relative humidity, and absolute humidity.

Property	**Description**
Dataset	UCI Air Quality
Original file	AirQualityUCI.csv
Original observations	9,358
Original attributes	15
Data type	Time-series environmental measurements
Measurement frequency	Hourly
Collection period	March 2004 – April 2005
Collection location	Urban area of Italy
Prediction task	Regression
Target variable	CO(GT)
Development environment	Google Colab / Python

The dataset contains missing or invalid sensor readings represented by the value -200. These values must be identified and handled appropriately before statistical analysis and model training.

2.1 Important Dataset Attributes

Attribute	Description
Date	Date of observation
Time	Time of observation
CO(GT)	Reference carbon monoxide concentration
PT08.S1(CO)	Sensor response associated with CO
NMHC(GT)	Non-methane hydrocarbons concentration
C6H6(GT)	Benzene concentration
PT08.S2(NMHC)	Sensor response associated with NMHC
NOx(GT)	Nitrogen oxides concentration
PT08.S3(NOx)	Sensor response associated with NOx
NO2(GT)	Nitrogen dioxide concentration
PT08.S4(NO2)	Sensor response associated with NO_2
PT08.S5(O3)	Sensor response associated with ozone-related measurements
T	Temperature
RH	Relative humidity
AH	Absolute humidity

The exact units and descriptions of individual variables should be interpreted according to the original dataset documentation.

2.2 Target Variable Definition

The target variable selected for the regression experiment is CO(GT).

The model uses other available numerical features to estimate the carbon monoxide concentration. Since the target is continuous, the problem is treated as a regression task rather than a classification task.

The target is not divided into discrete classes. The model produces a numerical estimate of CO(GT).

Important: This project estimates carbon monoxide concentration from available measurements. It does not directly forecast a future CO value.

3. EXPLORATORY DATA ANALYSIS

Exploratory Data Analysis was performed to understand the dataset's structure, quality, distributions, and relationships before developing the deep learning model.

The analysis includes inspecting the dataset, examining numerical summaries, identifying missing values, checking duplicate records, analysing potential outliers, and studying relationships between pollutant variables.

3.1 Data Inspection

The dataset was loaded into a Pandas DataFrame using Python. The original file uses a semicolon delimiter and a comma decimal separator, which were handled during data loading.

The initial inspection included:

- Displaying the first few records.
- Checking the number of rows and columns.
- Examining feature names and data types.
- Calculating descriptive statistics.
- Identifying completely empty columns.
- Checking missing-value counts and duplicate records.

Completely empty columns were removed from the working DataFrame.

The actual number of rows and columns remaining after cleaning should be recorded from the notebook output.

3.2 Missing-Value Analysis

The dataset represents missing or invalid sensor measurements using -200. These values were replaced with NaN so they could be identified as missing values.

Missing-value handling is important because invalid measurements can distort descriptive statistics and affect model training.

For the model pipeline, a median imputer was fitted using the training data and applied to the validation and test data. This helps prevent the preprocessing procedure from using information calculated from the held-out datasets.

The final report should record the actual missing-value counts before and after preprocessing.

3.3 Duplicate-Record Analysis

Duplicate records were checked to identify repeated observations. Removing duplicate rows can reduce unnecessary repetition in the dataset.

However, the number of duplicate rows should be reported from the actual notebook output rather than assumed.

3.4 Outlier Analysis

Outliers were examined using descriptive statistics and boxplots. An outlier may represent an unusually high pollutant concentration, an instrument issue, or a genuine environmental event.

The Interquartile Range (IQR) can be used to flag potential outliers:

$IQR = Q3 - Q1$

Lower bound $= Q1 - 1.5 \times IQR$

Upper bound $= Q3 + 1.5 \times IQR$

Observations outside these limits are potential statistical outliers. They should not automatically be removed because extreme pollutant measurements may contain useful environmental information.

3.5 Date and Time Processing

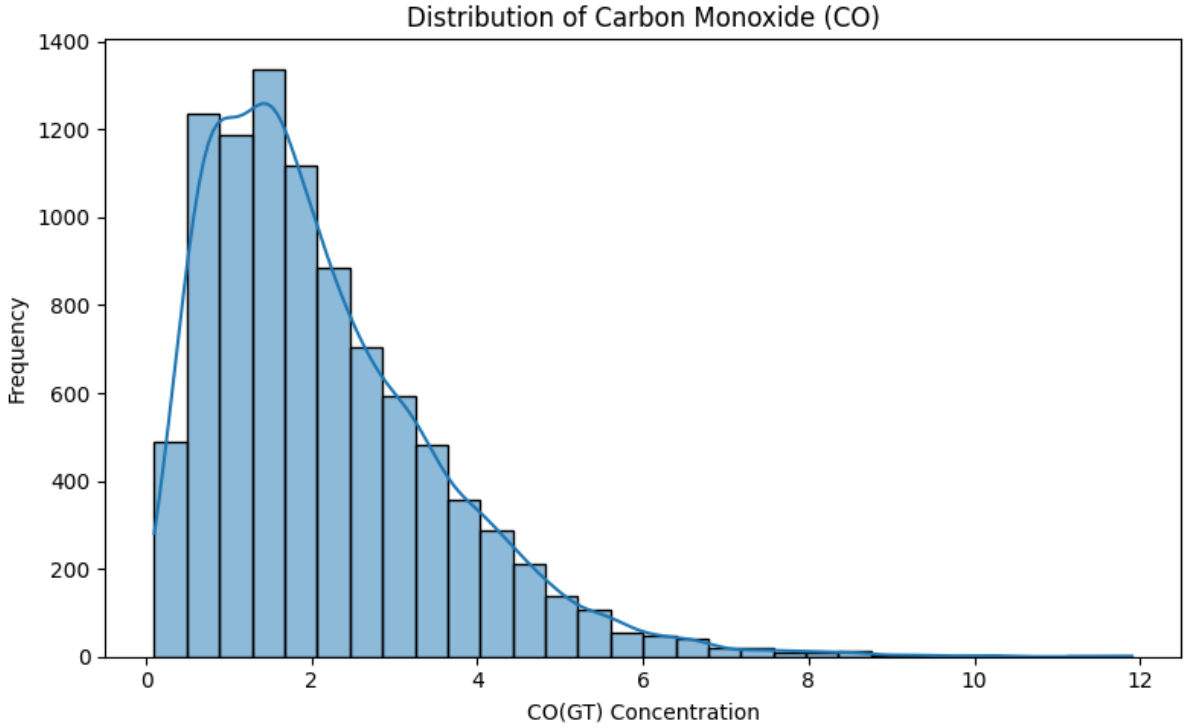
The Date and Time attributes were combined into a datetime variable. This enabled chronological sorting and analysis of hourly changes in carbon monoxide concentration.

Date and time information is particularly useful when examining environmental measurements because pollutant levels can vary across different periods.

3.6 Visualization Results

The following visualizations were created to understand the dataset and investigate patterns among the variables.

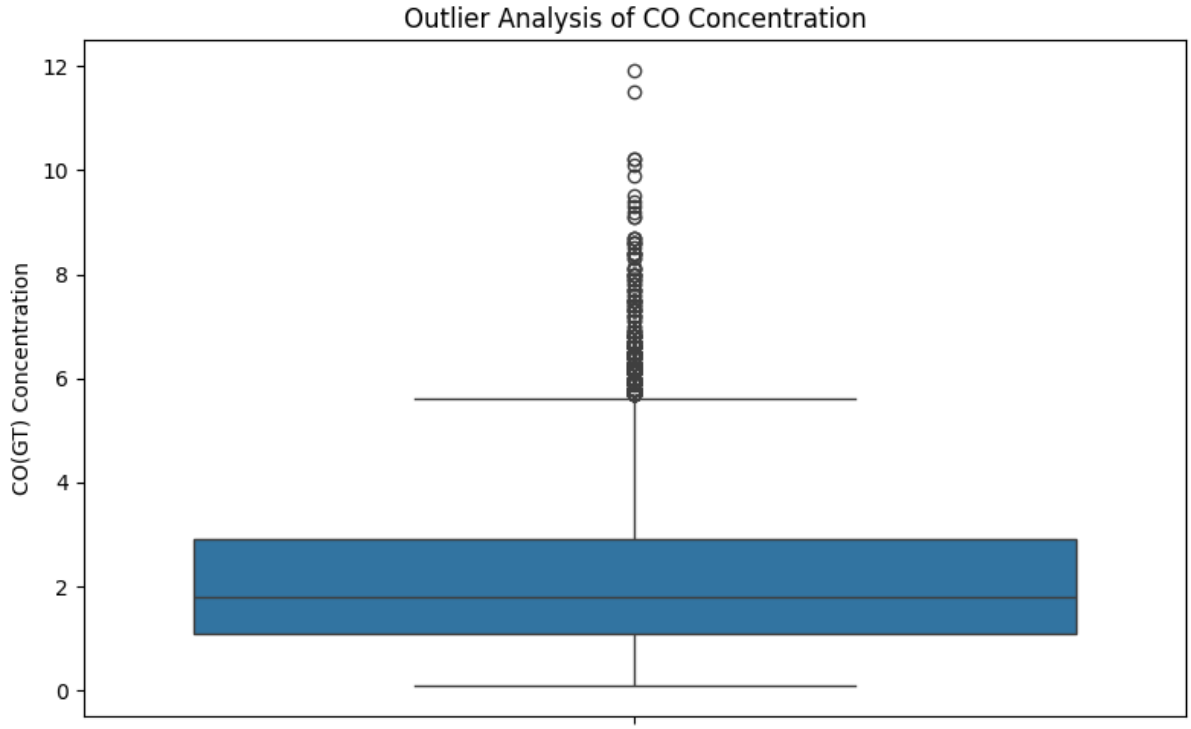
Figure 1: Histogram of CO(GT)



The histogram displays the frequency distribution of carbon monoxide measurements. It helps identify common concentration ranges, the spread of observations, and possible skewness.

If the distribution is right-skewed, it indicates that a large proportion of observations occur at lower concentrations, while fewer observations occur at higher concentrations. The actual distribution should be described based on the generated histogram.

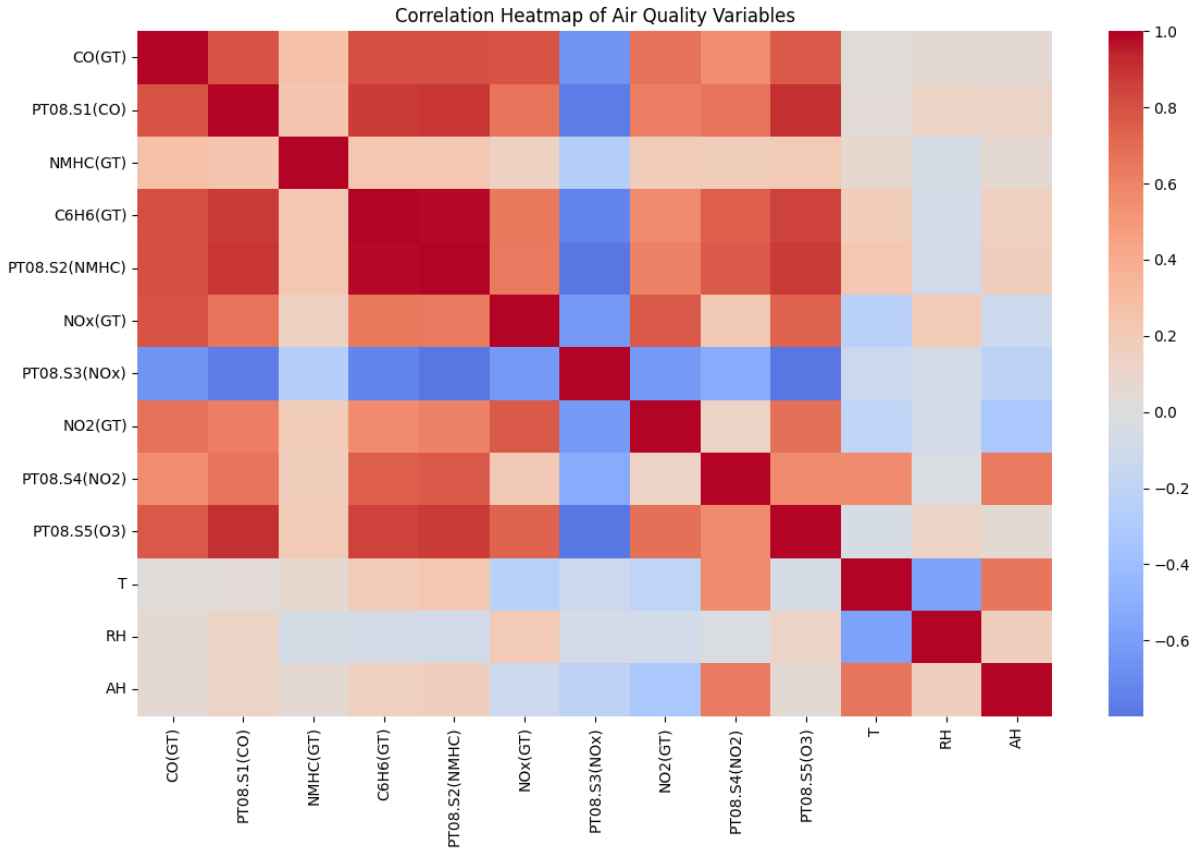
Figure 2: Boxplot of CO(GT)



The boxplot summarizes the median, interquartile range, and potential outliers of carbon monoxide measurements.

The visualization helps identify unusual observations and understand the variability of CO(GT). Potential outliers should be investigated rather than automatically removed.

Figure 3: Correlation Heatmap

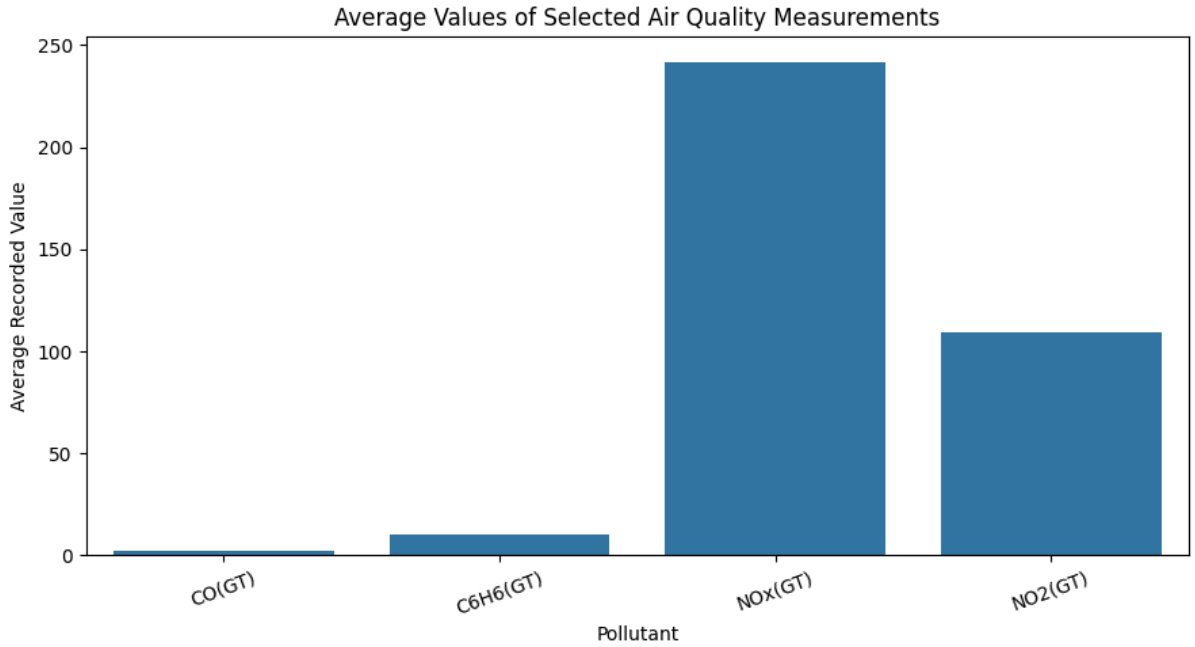


The correlation heatmap presents pairwise linear correlations among numerical attributes.

Positive correlations indicate that two variables tend to increase together, whereas negative correlations indicate that one variable tends to decrease as the other increases.

This visualization helps identify features associated with carbon monoxide concentration. However, correlation does not establish causation.

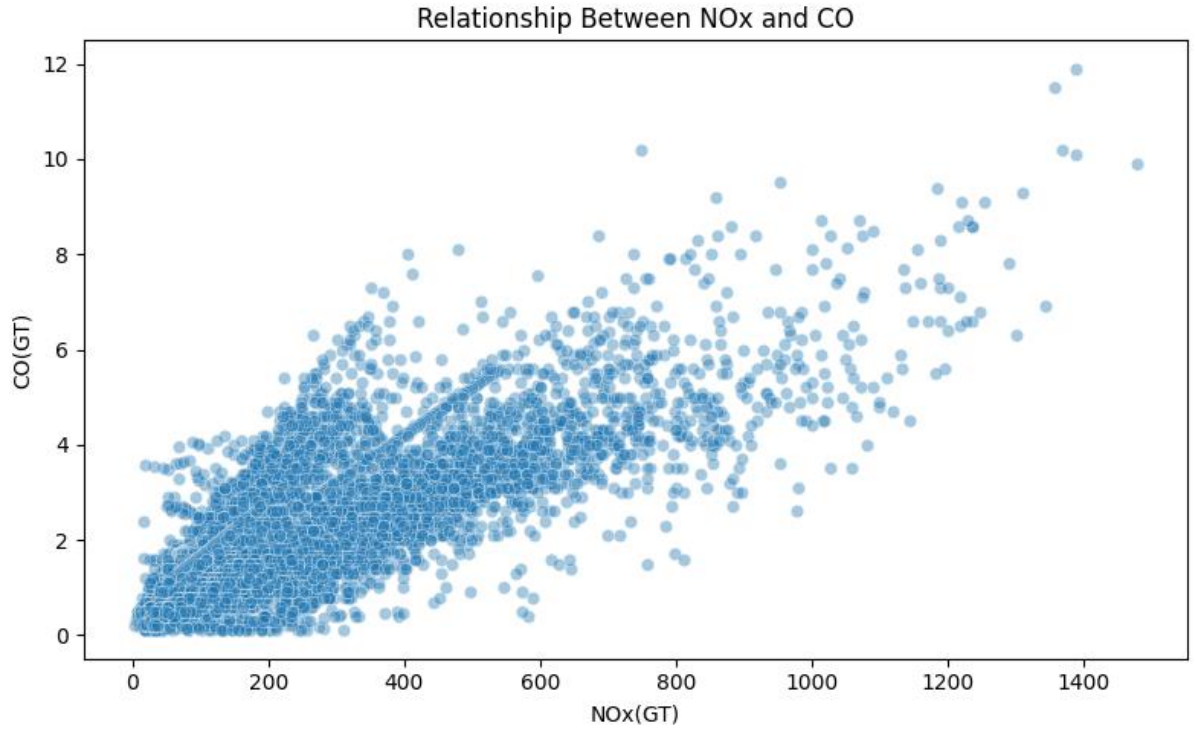
Figure 4: Comparison of Selected Pollutants



This visualization compares selected pollutant variables, including CO(GT), C6H6(GT), NOx(GT), and NO2(GT).

It provides an overview of the pollutant measurements in the dataset. Since the variables may have different units and numerical scales, the chart should be interpreted carefully rather than used to directly compare their health impacts.

Figure 5: NOx(GT) versus CO(GT)

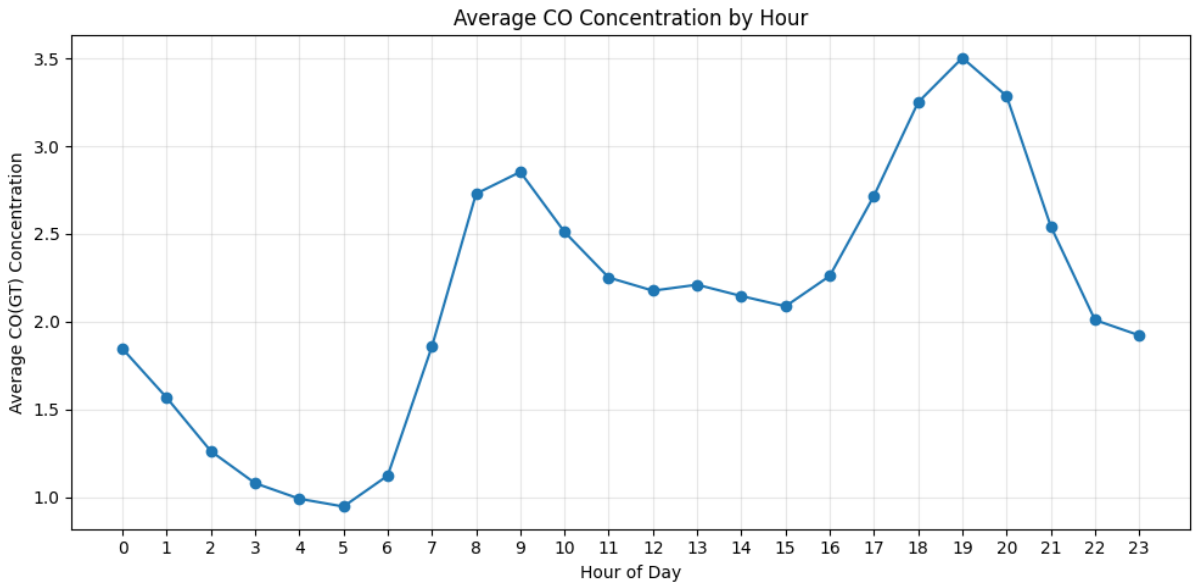


The scatter plot investigates the relationship between nitrogen oxides and carbon monoxide concentrations.

An upward pattern would indicate a positive association between the two variables. The plot can also reveal clusters, unusual observations, and possible nonlinear relationships.

The presence of an association does not establish that one pollutant causes a change in another.

Figure 6: Hourly CO(GT) Trend



The hourly trend chart displays average carbon monoxide concentration across different hours of the day.

This visualization helps explore whether CO measurements vary throughout the day. Changes may be associated with traffic patterns, environmental conditions, or other factors, but additional evidence is required to establish the reasons for any observed pattern.

3.7 Seven Stages of Visualization

The visualization workflow can be described using seven stages.

Stage	**Application in this project**
1. Acquire	Obtain the Air Quality dataset from the UCI Machine Learning Repository.
2. Parse	Load the dataset and interpret delimiters, decimal separators, columns, and data types.
3. Filter	Select valid observations and relevant numerical features for analysis.
4. Mine	Calculate descriptive statistics, correlations, pollutant summaries, and hourly averages.
5. Represent	Create histograms, boxplots, heatmaps, scatter plots, and trend charts.
6. Refine	Improve titles, axis labels, legends, layout, and readability.

7. Interact	Use optional interactive visualizations to explore observations and relationships.

These stages organize the visualization process from obtaining raw data to communicating useful analytical findings.

4. DATA PREPROCESSING AND EXPERIMENTAL DESIGN

Data preprocessing prepares the dataset for deep learning. The workflow includes handling invalid values, checking duplicates, creating a chronological order, selecting numerical features, imputing missing values, and standardizing the inputs.

4.1 Handling Invalid Values

The value -200 was treated as a missing-value indicator in the dataset. It was replaced with NaN before further analysis.

This step prevents the missing-value code from being treated as a genuine pollutant concentration.

4.2 Missing-Value Imputation

Median imputation was used in the model preprocessing pipeline. The imputer was fitted using the training data only and then applied to the validation and test sets.

The median is less sensitive to extreme observations than the mean, making it a useful baseline imputation strategy for numerical data.

4.3 Feature Selection

The target variable CO(GT) was separated from the input features.

The input matrix was constructed using the available numerical attributes other than the target. Date and time fields were used to establish chronological ordering rather than being included directly as numerical model inputs.

Only the features actually selected by the notebook should be listed in the final implementation description.

4.4 Chronological Train-Validation-Test Split

The data was arranged chronologically and divided into three subsets.

Subset	Approximate proportion	Purpose
Training	70%	Fit the model and preprocessing transformations
Validation	15%	Monitor training and select model configurations
Test	15%	Evaluate final performance on held-out observations

A chronological split is suitable for this dataset because the observations represent measurements collected over time.

The training set is used to fit the imputer and scaler. The same fitted transformations are then applied to the validation and test sets.

Preprocessing limitation: If interpolation was applied to the entire dataset before splitting, it may introduce information leakage across time boundaries. A rigorous final experiment should perform missing-value handling within the chronological splits or use a leakage-safe pipeline.

4.5 Feature Scaling

StandardScaler was used to standardize the numerical input features.

For a feature x, standardization is expressed as:

$z = (x - \mu) / \sigma$

where μ represents the training-set mean and σ represents the training-set standard deviation.

Scaling is useful for neural networks because the input features may have different ranges. The scaler was fitted on the training data and then applied to the validation and test data.

5. DEEP LEARNING MODEL DEVELOPMENT

5.1 Model Selection

A Multilayer Perceptron (MLP) was selected for the regression task. An MLP is a feed-forward neural network that learns relationships between input features and a continuous target variable.

The model consists of an input layer, two hidden layers, and a single output neuron.

5.2 Model Architecture

The implemented baseline architecture is summarized below.

Component	**Configuration**	**Purpose**
Input layer	Numerical feature vector	Receives preprocessed features
Hidden layer 1	Dense, 64 neurons, ReLU	Learns nonlinear feature combinations
Hidden layer 2	Dense, 32 neurons, ReLU	Learns higher-level representations
Output layer	Dense, 1 neuron, linear output	Estimates CO(GT)
Optimizer	Adam	Updates model weights
Loss function	Mean Squared Error	Minimizes squared prediction errors
Evaluation metric	Mean Absolute Error	Measures average absolute prediction error

The input layer accepts the preprocessed numerical features. The hidden layers use ReLU activation to learn nonlinear relationships. The output layer contains one neuron because the task requires a single continuous prediction.

5.3 Training Configuration

Parameter	Configuration
Model type	MLP regression
Hidden layers	64 and 32 neurons
Activation function	ReLU
Optimizer	Adam
Learning rate	0.001
Loss function	Mean Squared Error
Training metric	Mean Absolute Error
Batch size	32
Maximum epochs	50
Early stopping	Validation loss, patience 8

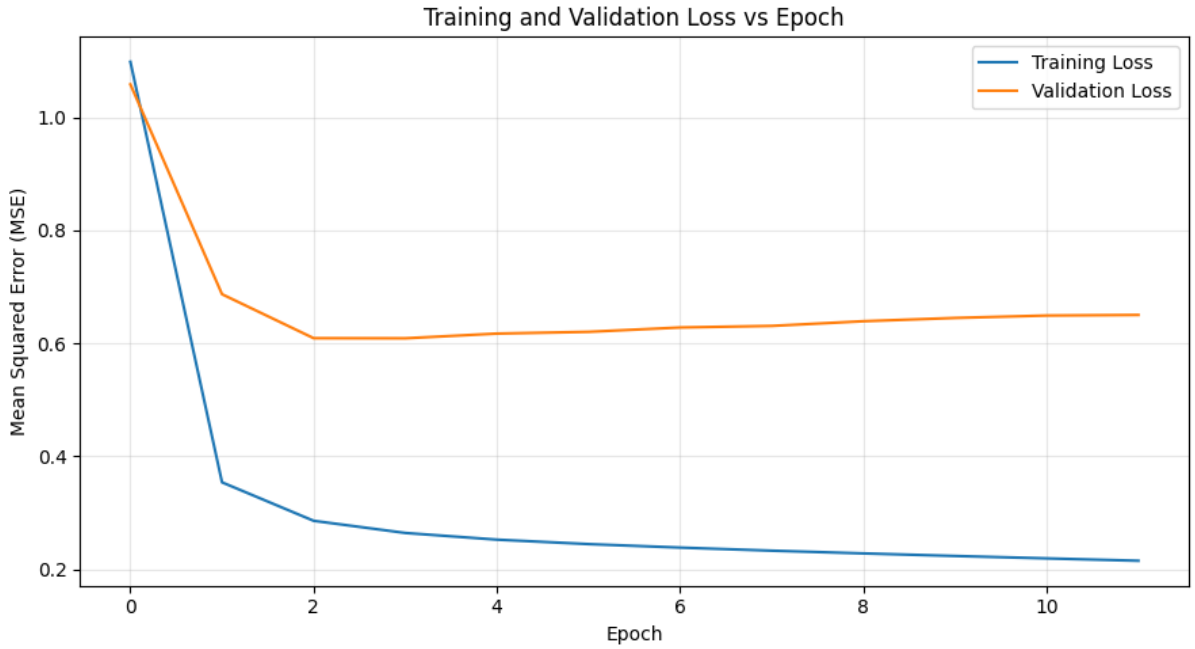
Early stopping monitors the validation loss and restores the weights from the best observed epoch. This helps avoid unnecessary training and may reduce overfitting.

5.4 Model Training

The model was trained using the training dataset. Validation data was used to monitor learning performance and support early stopping.

During training, the neural network updated its weights to reduce the difference between predicted and actual carbon monoxide concentrations.

The training and validation loss values were recorded across epochs.



The learning curve helps assess the training process. A decreasing training loss generally indicates that the model is learning from the training data. If validation loss increases while training loss continues to decrease, the model may be overfitting.

The actual training behaviour should be described based on the generated learning curve.

6. MODEL EVALUATION AND RESULTS

The trained MLP regression model was evaluated on the held-out test dataset. Three standard regression metrics were used: Mean Absolute Error, Root Mean Squared Error, and the coefficient of determination.

6.1 Evaluation Metrics

Mean Absolute Error (MAE)

MAE measures the average absolute difference between actual and predicted values.

A lower MAE indicates that the predictions are, on average, closer to the actual target values.

Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared prediction error. Because errors are squared before averaging, larger errors have a greater influence on this metric.

Coefficient of Determination (R^2)

R^2 measures how much of the variation in the target is explained by the model relative to the standard mean-prediction baseline.

An R^2 value closer to 1 generally indicates a better fit on the evaluated dataset. R^2 is not classification accuracy.

6.2 Test Results

Metric	Measured result
Test MAE	0.4542
Test RMSE	0.6097
Test R^2	0.7861

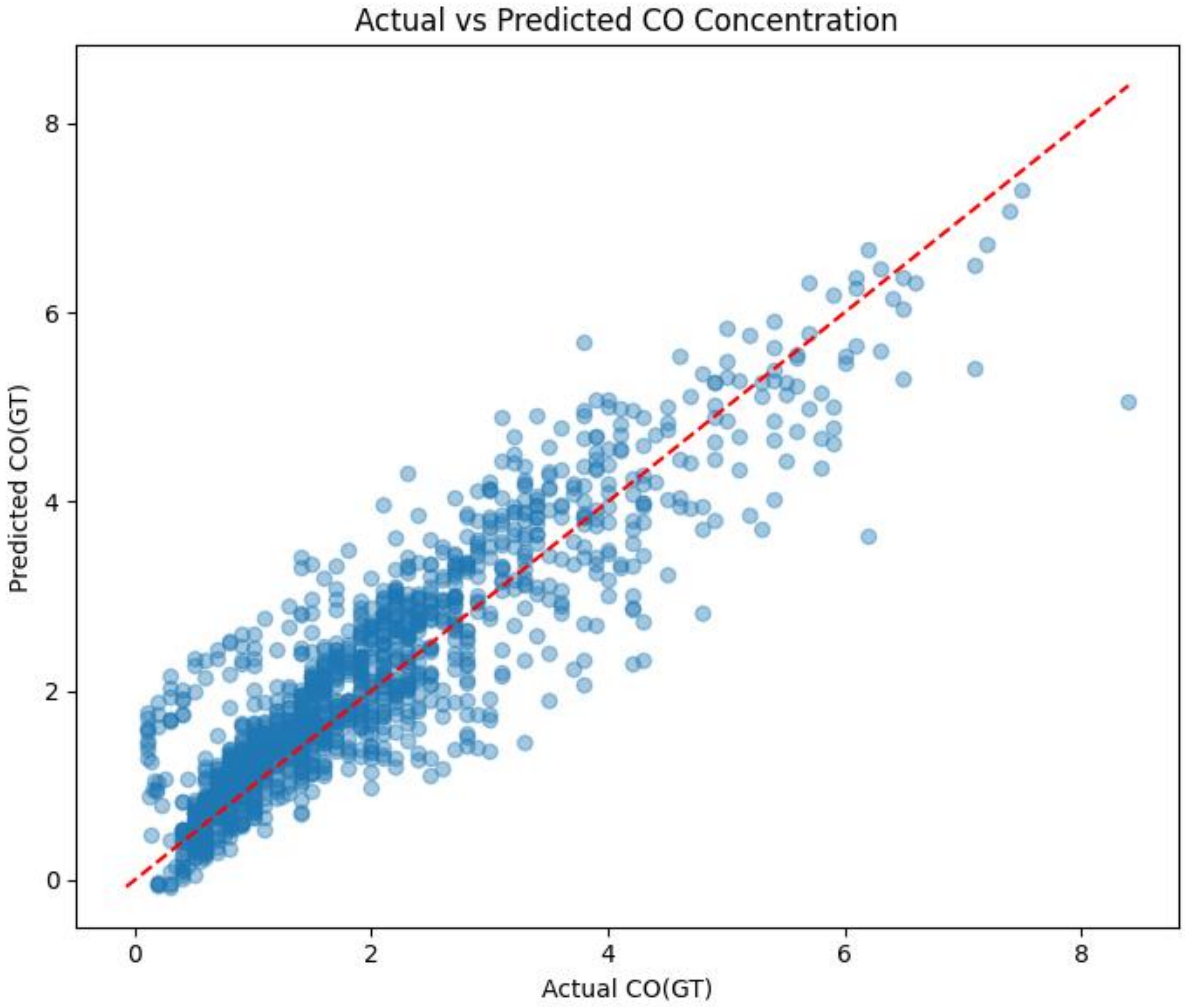
The model achieved a test MAE of 0.4542 and an RMSE of 0.6097. These values represent the average absolute error and the root mean squared error in the target variable's units.

The R^2 score of 0.7861 indicates that the model explains approximately 78.61% of the observed variation in carbon monoxide concentration on the test set relative to the standard baseline.

The results suggest that the model captures a substantial proportion of the variation in CO(GT), although some prediction errors remain.

The values should be interpreted in the context of the dataset, the selected features, the preprocessing procedure, and the chronological test split.

6.3 Actual versus Predicted Values

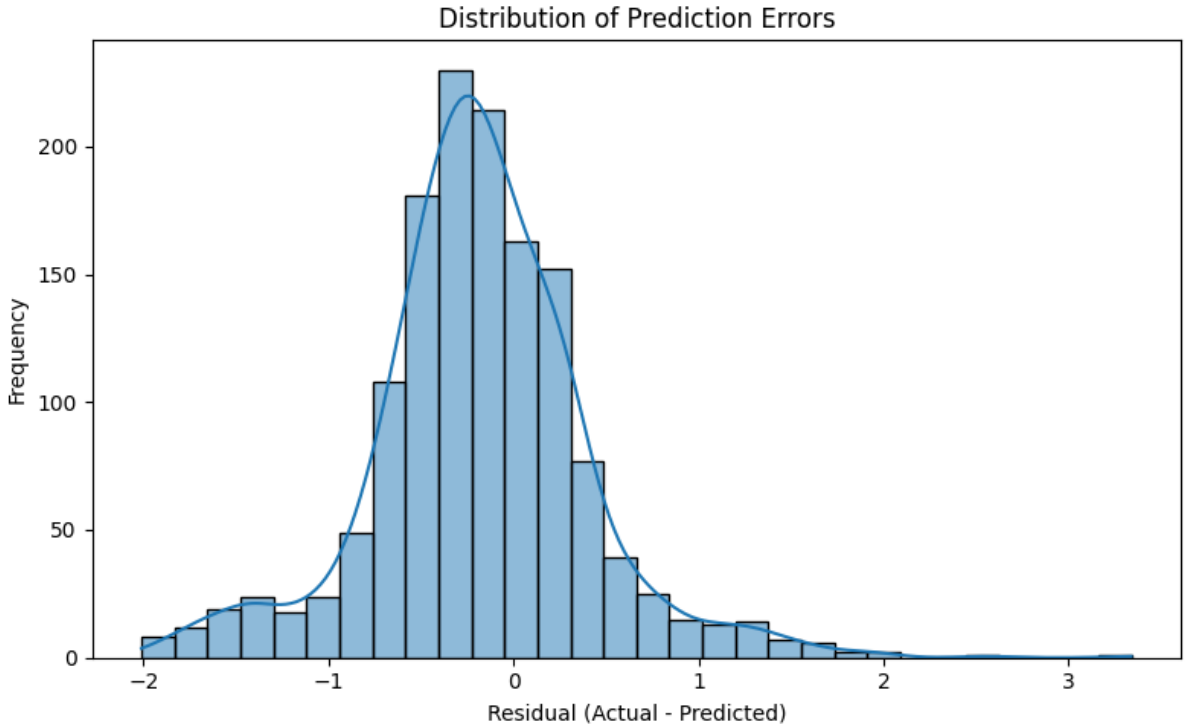


The actual-versus-predicted scatter plot compares the measured carbon monoxide concentrations with the values estimated by the MLP model.

A reference diagonal represents perfect predictions. Points close to the diagonal indicate smaller prediction errors, whereas points farther away indicate larger errors.

The plot helps identify whether the model systematically underestimates or overestimates particular ranges of CO concentration.

6.4 Residual Error Distribution

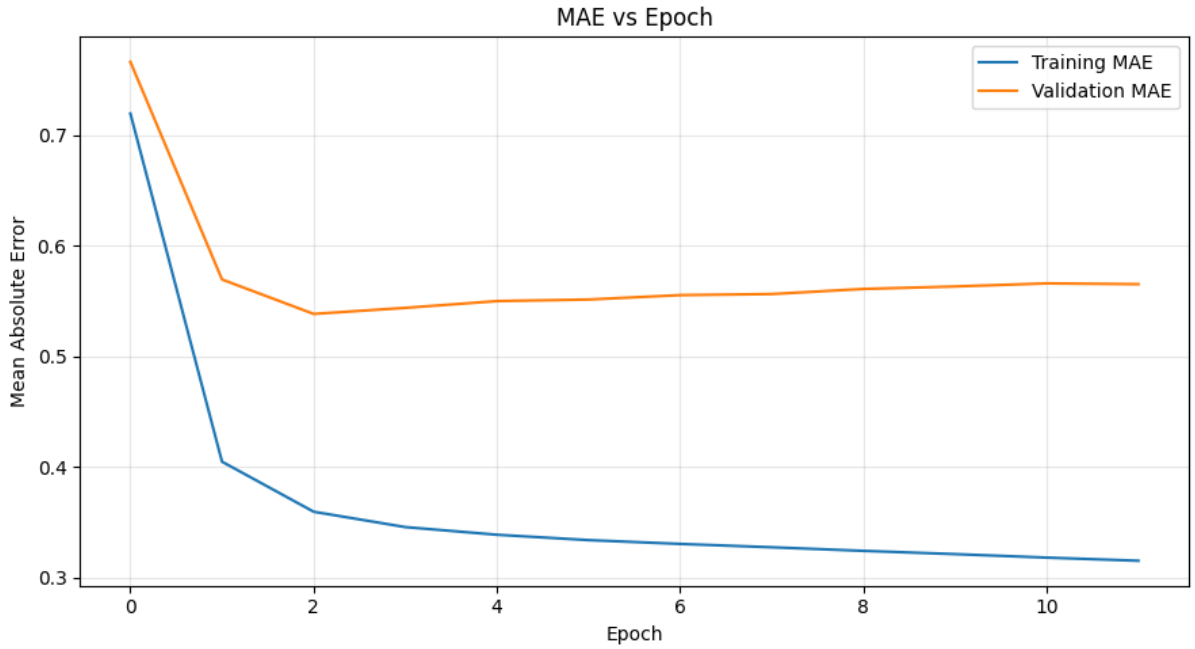


Residuals represent the differences between actual and predicted values.

The residual distribution helps assess the spread and direction of prediction errors. A distribution centred near zero suggests limited overall bias, while skewness or extreme residuals may indicate systematic prediction errors.

The actual residual histogram should be used to describe the model's error pattern.

6.5 MAE versus Epoch



The MAE learning curve displays the changes in training and validation mean absolute error across epochs.

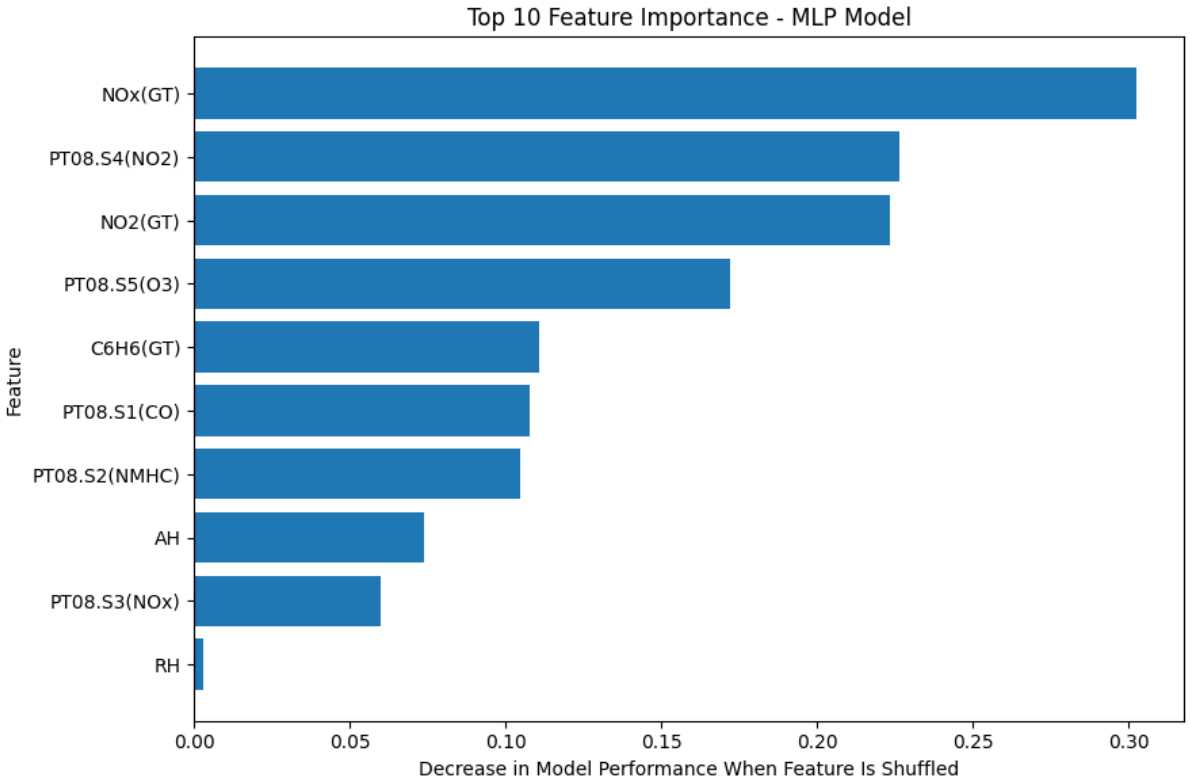
Comparing the two curves helps determine whether prediction error improves during training and whether a substantial gap develops between training and validation performance.

7. FEATURE IMPORTANCE AND HYPERPARAMETER TUNING

7.1 Feature Importance Analysis

Permutation importance can be used to examine how much the trained model relies on individual input features.

The method shuffles one feature at a time and measures the resulting change in model performance. If shuffling a feature increases the prediction error, that feature may be important to the model's predictions.



The final feature ranking should be inserted from the actual permutation-importance output produced by the notebook.

Features with larger positive importance values may have a stronger contribution to predictive performance under this evaluation procedure. However, correlated features can influence each other's importance scores, and permutation importance does not establish causation.

Actual top features: [Insert the feature names and importance values from the notebook output.]

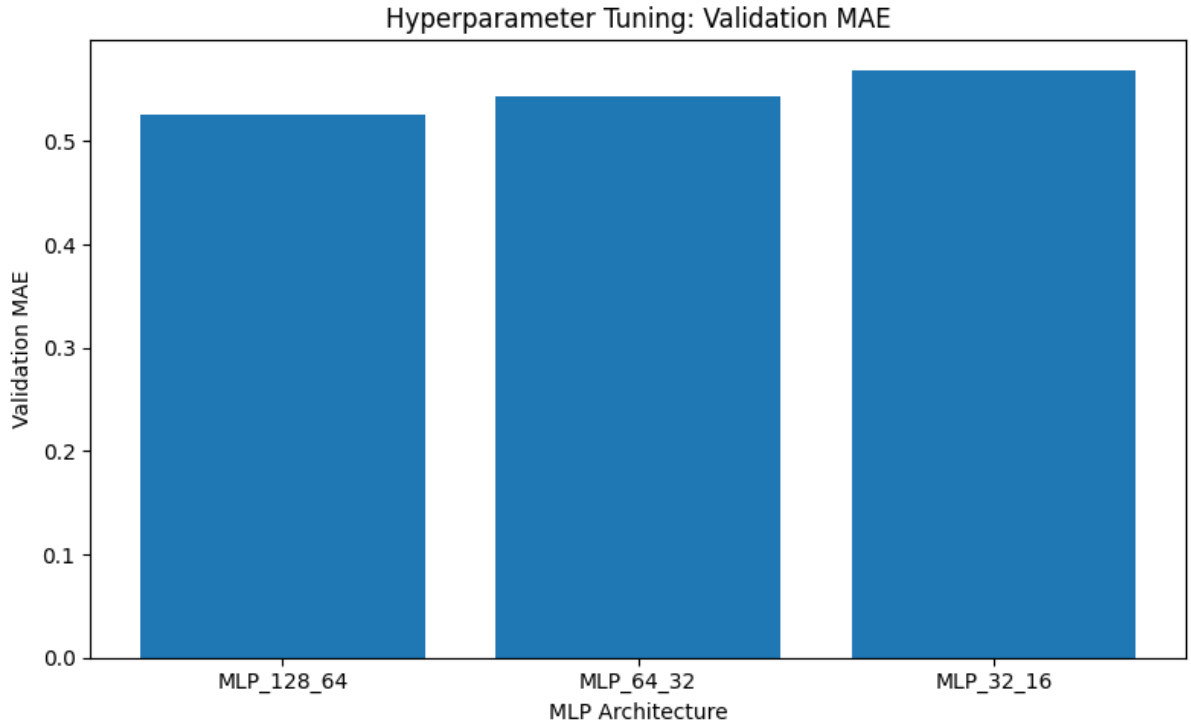
7.2 Hyperparameter Tuning

Hyperparameter tuning explores alternative neural-network configurations to identify settings that perform well on validation data.

The following hidden-layer configurations were considered for comparison:

Configuration	Hidden-layer structure
Model A	32, 16 neurons
Model B	64, 32 neurons
Model C	128, 64 neurons

The configurations can be compared using validation MAE and validation MSE under the same preprocessing and evaluation procedure.



The configuration with the lowest validation error may be preferred when using an error-minimization selection criterion. The selected configuration and its validation results should be reported using the actual tuning output.

Best configuration: [Insert the configuration selected from the actual validation results.]

The test set should remain reserved for final evaluation and should not be repeatedly used to select hyperparameters.

8. CONCLUSION

This project demonstrates an end-to-end data science workflow for air quality analysis and carbon monoxide estimation using exploratory data analysis, data preprocessing, visualization, and deep learning.

The UCI Air Quality dataset was examined to understand pollutant distributions, potential outliers, correlations, and hourly variations. Missing-value handling, duplicate checks, numerical imputation, feature scaling, and chronological data splitting were used to prepare the dataset for modelling.

A Multilayer Perceptron regression model was developed to estimate CO(GT) from other numerical features. The model achieved a test MAE of 0.4542, an RMSE of 0.6097, and an R^2

score of 0.7861. These results indicate that the model explains approximately 78.61% of the variation in the target on the evaluated test set.

The project highlights the importance of EDA, appropriate preprocessing, validation-based model monitoring, and multiple evaluation metrics. Further feature-importance analysis and hyperparameter comparisons can help identify opportunities to improve the model.

The model should be considered an educational analysis prototype. Its predictions are estimates based on the available dataset and should not be interpreted as future pollution forecasts or substitutes for official air quality monitoring.

9. LIMITATIONS AND FUTURE SCOPE

9.1 Limitations

- The dataset represents measurements collected in a particular urban area of Italy during 2004–2005.
- Missing and invalid sensor readings may affect the reliability of the analysis.
- The model's performance depends on the selected features, preprocessing methods, and chronological split.
- Data leakage may occur if missing values are interpolated using observations across training and test boundaries.
- A high R^2 score on this dataset does not guarantee equivalent performance in other locations or periods.
- The model estimates CO concentration from available measurements and does not directly predict future air quality.
- Further external validation is required before applying the model in a real-world environmental monitoring system.

9.2 Future Enhancements

- Compare the MLP model with Random Forest, Gradient Boosting, and other regression algorithms.
- Explore systematic hyperparameter optimization to identify suitable neural-network configurations.
- Improve missing-value treatment using time-aware methods while preventing information leakage.
- Develop LSTM or GRU models for explicit time-series forecasting.

- Evaluate the model using data from additional locations and time periods.
- Integrate live sensor measurements into an air quality dashboard.
- Use explainability techniques to better understand model behaviour.
- Monitor model performance when sensor characteristics or environmental conditions change.

10. REFERENCES

1. S. De Vito, E. Massera, M. Piga, L. Martinotto, and G. Di Francia. (2008). "On Field Calibration of an Electronic Nose for Benzene Estimation in an Urban Pollution Monitoring Scenario." *Sensors and Actuators B: Chemical*, 129(2), 750–757. Consult the original dataset documentation for the complete dataset citation.
2. UCI Machine Learning Repository. (n.d.). *Air Quality Dataset*. Available at: https://archive.ics.uci.edu/dataset/360/air+quality
3. F. Pedregosa et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html
4. TensorFlow. (n.d.). *Keras Documentation*. Available at: https://keras.io/
5. pandas Development Team. (n.d.). *pandas Documentation*. Available at: https://pandas.pydata.org/docs/
6. J. D. Hunter. (2007). "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), 90–95. DOI: https://doi.org/10.1109/MCSE.2007.55

APPENDIX A: SOURCE CODE AND IMPLEMENTATION

The complete implementation is provided in the Google Colab/Jupyter notebook:

Air_Quality_Analysis_Deep_Learning.ipynb

The notebook contains:

- Dataset downloading and loading.
- Data inspection and preprocessing.
- Missing-value handling and duplicate checks.
- Exploratory data analysis and visualizations.
- Chronological train-validation-test splitting.
- Numerical imputation and feature scaling.
- MLP model creation and training.
- Training and validation learning curves.
- Test-set evaluation using MAE, RMSE, and R^2.
- Actual-versus-predicted analysis and residual visualization.
- Feature importance and model configuration comparisons, if executed.

The notebook should be submitted along with the report to make the analysis reproducible.

APPENDIX B: GITHUB REPOSITORY

The project can be organized using the following structure:

Air-Quality-Analysis-Deep-Learning/

- Air_Quality_Analysis_Deep_Learning.ipynb
- Project_Report.pdf
- README.md
- requirements.txt
- visualizations/
- results/

The README file should describe the project objective, dataset source, required Python libraries, execution steps, and model evaluation results.

GitHub repository: Air_Quality_Analysis_Deep_Learning

Dataset URL: https://archive.ics.uci.edu/dataset/360/air+quality